Deploying Flask on AWS Elastic Beanstalk

In this section we'll deploy our Flask application using AWS Elastic Beanstalk. AWS Elastic Beanstalk is an easy-to-use service for deploying and scaling web applications and services.

We'll assume that you already have an active AWS account and AWS CLI setup in your development machine or else you can use AWS docs on that.

To create the application environment and deploy the application, initialize your EB CLI repository with the eb init command.

```
$ eb init -p python-2.7 flask-app --region <your_region>
```

Note For the list of regions, refer to this guide: https://docs.aws.amazon.com/general/latest/gr/rande.html

---

You should see the following response in your terminal.

```
$ Application flask-app has been created.
```

The preceding command creates a new application named flask-app and configures your local repository to create environments with the latest Python 2.7.

Next run eb init again to configure a keypair for SSH login.

Next we'll create an environment and deploy your application to it with eb create.

```
$ eb create
```

Next enter the environment name, DNS prefix, and load balancer type which will eliminate the need to expose the web server to the world.

```
Kunals-MacBook-Pro:src kunalirelan$ eb create
Enter Environment name
(default is flask-app-dev):
Enter DNS CNAME prefix
(default is flask-app-dev):

Select a load balancer type
1) classic
2) application
(default is 3): 2
Creating application version archive "app-190428_235315".
```

Figure 6-5. *eb create*

Now it shall take around 5 minutes to be deployed. Once it is deployed, we just need to configure a few more details, which we'll directly do in the AWS web console.

```
Creating application version archive "app-190428_235315".
Uploading: [##################################################] 100% Done...
Environment details for: flask-app-dev
  Application name: flask-app
  Region: ap-south-1
  Deployed Version: app-190428_235315
  Environment ID: e-kmqmn6dqdg
  Platform: arn:aws:elasticbeanstalk:ap-south-1::platform/Python 2.7 running on 64bit Amazon Linux/2.8.2
  Tier: WebServer-Standard-1.0
  CNAME: flask-app-dev.ap-south-1.elasticbeanstalk.com
  Updated: 2019-04-28 18:24:19.801000+00:00
Printing Status:
INFO: createEnvironment is starting.
INFO: Using elasticbeanstalk-ap-south-1-540270294496 as Amazon S3 storage bucket for environment data.
INFO: Environment health has transitioned to Pending. Initialization in progress (running for 11 seconds). There are no instances.
INFO: Created target group named: arn:aws:elasticloadbalancing:ap-south-1:540270294496:targetgroup/awseb-AWSEB-1JU4GMVD0YF6G/841805aeb695b1a6
INFO: Created security group named: sg-038ca52923ice7c6a
INFO: Created security group named: awseb-e-kmqmn6dqdg-stack-AWSEBSecurityGroup-18JY1ECMKQVTE
INFO: Created Auto Scaling launch configuration named: awseb-e-kmqmn6dqdg-stack-AWSEBAutoScalingLaunchConfiguration-TOLAY2OMQX2I
INFO: Created Auto Scaling group named: awseb-e-kmqmn6dqdg-stack-AWSEBAutoScalingGroup-COUT3QGHBQUM
INFO: Waiting for EC2 instances to launch. This may take a few minutes.
INFO: Created Auto Scaling group policy named: arn:aws:autoscaling:ap-south-1:540270294496:scalingPolicy:64967d8f-41de-4dc5-a6d8-61b8304bef9:autoScalingGroupName/awseb-e-kmqmn6dqdg-stack-AWSEBAutoScaling
Group-COUT3QGHBQUM:policyName/awseb-e-kmqmn6dqdg-stack-AWSEBAutoScalingScaleUpPolicy-X0MUFHBEJCOJ
INFO: Created Auto Scaling group policy named: arn:aws:autoscaling:ap-south-1:540270294496:scalingPolicy:e0c30715-269f-48fa-8e19-ff61bece4075:autoScalingGroupName/awseb-e-kmqmn6dqdg-stack-AWSEBAutoScaling
Group-COUT3QGHBQUM:policyName/awseb-e-kmqmn6dqdg-stack-AWSEBAutoScalingScaleDownPolicy-QTTSF283K5BW
INFO: Created CloudWatch alarm named: awseb-e-kmqmn6dqdg-stack-AWSEBCloudwatchAlarmHigh-1M6XPOF8MPYV
INFO: Created CloudWatch alarm named: awseb-e-kmqmn6dqdg-stack-AWSEBCloudwatchAlarmLow-5MCVB8JG6HAA
INFO: Added instance [i-02aa75bdccbc30a56] to your environment.
INFO: Created load balancer named: arn:aws:elasticloadbalancing:ap-south-1:540270294496:loadbalancer/app/awseb-AWSEB-17287FNCC7YQR/afe61e56d3215f2c
INFO: Created Load Balancer listener named: arn:aws:elasticloadbalancing:ap-south-1:540270294496:listener/app/awseb-AWSEB-17287FNCC7YQR/afe61e56d3215f2c/426b1f4f0f661947
ERROR: Your WSGIPath refers to a file that does not exist.
INFO: Application available at flask-app-dev.ap-south-1.elasticbeanstalk.com.
INFO: Successfully launched environment: flask-app-dev
```

Figure 6-6. *eb create success*

Once the application finishes deploying, you should see a similar output like in the preceding figure. Next log on to the AWS web console, and open Elastic Beanstalk Configuration tab.

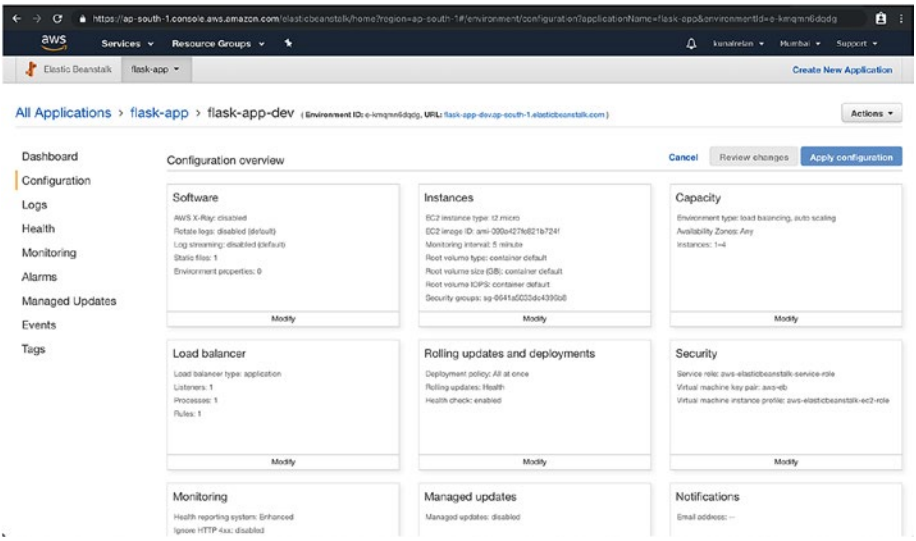

Figure 6-7. *Elastic Beanstalk Application Configuration*

Next click on modify; on the software tab and inside container options, update the WSGIPath to run.py.

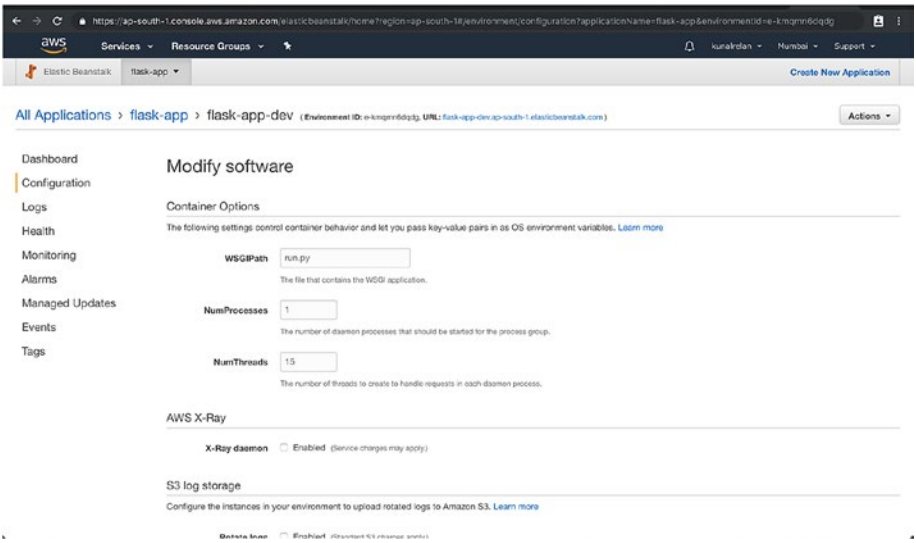

Figure 6-8. *WSGIpath set*

Now scroll down, and in environment variable, supply WORK_ENV and set it to Prod so that our application runs in production mode. Next click on apply and the application should reload and start working.

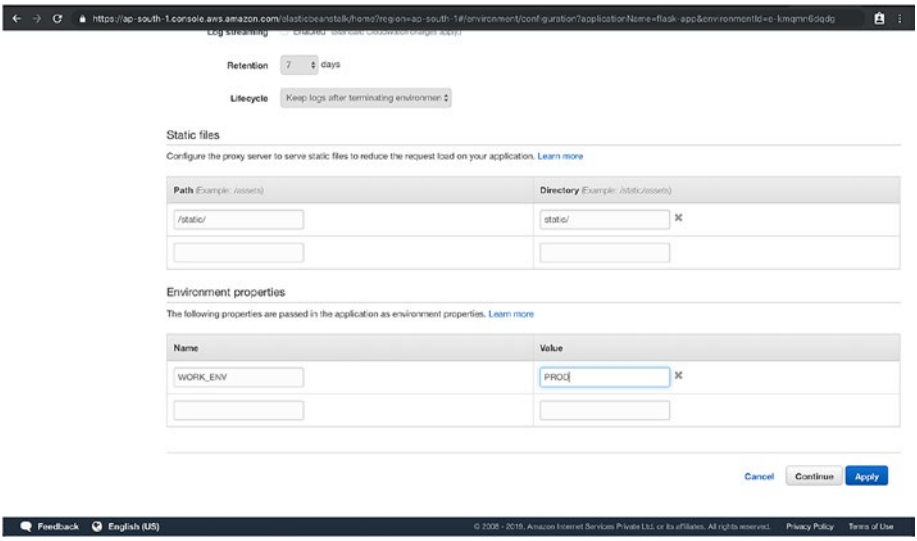

Figure 6-9. *Elastic Beanstalk Environment variables*

Now you can go back to dashboard to find the URL of the application, and it should be up and running.

Note In Elastic Beanstalk you can also configure and start a MySQL RDS server attached to the environment to run with the application, but it is out of the scope of this book.